

exercises.js — Fundamentos

Explicación detallada de la tabla de ejercicios y sus conexiones (GymBro App)

Es un archivo de **datos puros** (no un componente, no tiene JSX ni hooks) que actúa como la “tabla de ejercicios” de la base de datos simulada de la app. Junto con users.js y routines.js, forma la capa de datos mockeada de todo el proyecto.

1. Qué es conceptualmente

```
export const exercises = [
  { id: 1, name: '3/4 abdominales', muscleGroup: 'Abdomen', video: '...', description: '...' },
  ...
];
```

Es un **array de objetos** exportado, cada uno representando un ejercicio con 5 campos:

- **id**: identificador único numérico (clave primaria, en términos de base de datos).
- **name**: nombre visible del ejercicio.
- **muscleGroup**: categoría/grupo muscular ('Abdomen', 'Pecho', 'Pierna', 'Espalda', 'Hombro', 'Movilidad', 'Cardio') — se usa para filtrar/buscar.
- **video**: URL del video demostrativo (son placeholders https://tu-video.com/..., todavía no hay videos reales — esto es lo que consume ExerciseVideo.js).
- **description**: instrucciones de ejecución en texto.

Es un catálogo **fijo y compartido**: no pertenece a ningún usuario en particular, es el mismo listado de 12 ejercicios disponible para toda la app (a diferencia de users.js, donde cada usuario es distinto).

2. La función getExerciseById (líneas 88-90)

```
export function getExerciseById(exerciseId) {
  return exercises.find((exercise) => exercise.id === exerciseId);
}
```

Es un **helper de acceso** (lo que en una base de datos real sería un SELECT * FROM exercises WHERE id = ?). Usa .find(), que recorre el array y devuelve el **primer** elemento que cumple la condición, o undefined si ninguno coincide. Se exporta como función independiente en vez de obligar a cada pantalla a escribir exercises.find(...) a mano — mismo patrón que getUserById en users.js y getRoutineById en routines.js.

3. Con quién se conecta — el modelo de datos relacional simulado

Acá está la parte más importante para entender la arquitectura de datos de la app. Este proyecto simula una base de datos relacional con **tres tablas** (users, routines, exercises) que se referencian entre sí por id, sin usar ningún motor de base de datos real — todo son arrays de JS en memoria.

routines.js es la pieza que conecta todo. Mirá la forma de un item dentro de una rutina:

```
// dentro de routines.js
{
  id: 1,
  type: 'exercise',
```

```

    exerciseId: 2,      // <- NO guarda el ejercicio completo, solo su id
    series: '4',
    repetitions: '10',
    weight: '0 kg',
    comments: '',
  }
}

```

Una rutina (routines) tiene days → cada día tiene blocks → cada bloque tiene items → cada item, si es de type: 'exercise', **no contiene los datos del ejercicio en sí** (nombre, video, descripción), sino solo un **exerciseId** que apunta a exercises.js. Es exactamente el concepto de **clave foránea (foreign key)** de una base de datos relacional: routines no duplica la información de exercises, la referencia.

Esto separa dos responsabilidades distintas:

- **exercises.js**: “qué es este ejercicio” (nombre, cómo se hace, video) — dato genérico, reutilizable.
- **routines.js**: “cómo se combina este ejercicio en un plan específico” (cuántas series, repeticiones, peso, en qué bloque de qué día) — dato específico de cada rutina.

Si separaran los datos así, se evita duplicación: si mañana corrigen la descripción de “Sentadilla”, lo cambian en un solo lugar (exercises.js) y automáticamente se refleja en todas las rutinas que usan exerciseId: 5, sin tener que buscar y actualizar cada rutina que lo incluya.

4. Dónde se usa en las pantallas (los “JOIN” manuales)

Como los items de rutina solo tienen el exerciseId, cada pantalla que necesita **mostrar** el ejercicio completo tiene que hacer el equivalente a un JOIN: llamar getExerciseById(item.exerciseId) para “traer” los datos completos.

ver_dia_x_user.js (pantalla del alumno, ver el detalle de un día de su rutina)

```

import { getExerciseById } from '../data/exercises';
// ...
{getExerciseById(item.exerciseId)?.name}
// ...
onPress={() => showVideo(getExerciseById(item.exerciseId))}
onPress={() => showInstructions(getExerciseById(item.exerciseId))}

```

Por cada item de tipo ejercicio dentro de un bloque, busca el ejercicio completo para mostrar su nombre, y para abrir el modal de video o de instrucciones — el video={selectedExercise.video} que se le pasa a ExerciseVideo viene, en última instancia, de acá.

Nota el **optional chaining** ?.name: si getExerciseById no encontrara el ejercicio (por ejemplo, un exerciseId inválido), devolvería undefined, y undefined?.name no rompe la app (da undefined en vez de lanzar un error de “no se puede leer 'name' de 'undefined'”).

editar_rutina_trainer.js (pantalla del entrenador, armar/editar la rutina de un alumno)

```

import { exercises, getExerciseById } from '../data/exercises';
// ...
const ejerciciosFiltrados = exercises.filter((exercise) =>
  normalizar(`${exercise.name} ${exercise.muscleGroup}`).includes(
    normalizar(exerciseName)
  )
).slice(0, 3);
// ...
{getExerciseById(item.exerciseId)?.name}

```

Acá se usa **el array completo** (exercises), no solo el helper: cuando el entrenador escribe en el buscador de ejercicios para agregar uno a un bloque, filtra el catálogo completo por coincidencia de

texto en name + muscleGroup (usando una función normalizar para ignorar tildes/mayúsculas), y .slice(0, 3) limita el resultado a los primeros 3 matches para no saturar la pantalla con una lista larga. Cuando el entrenador elige uno, se guarda solo su id como exerciseId en el nuevo item de rutina (newExercise = { id: Date.now(), exerciseId: selectedExerciseId, ... }), reforzando el modelo de “guardar solo la referencia, no el objeto completo”.

mi_rutina_user.js también importa de exercises.js con el mismo propósito general (mostrar información derivada del catálogo dentro de la vista de rutina del alumno).

5. Resumen del flujo completo (útil para dibujar en la defensa)

```
users.js      -> cada user tiene un id_rutina
                |
routines.js   -> getRoutineById(id_rutina) trae la rutina completa
                | (days -> blocks -> items)
                | cada item de tipo 'exercise' solo tiene un exerciseId
                |
exercises.js  -> getExerciseById(exerciseId) trae nombre/video/descripcion
```

Es el mismo patrón de “IDs que referencian otras tablas” que ya se ve en AuthContext.js (user.id_rutina apuntando a routines.js), aplicado un nivel más adentro: item.exerciseId apuntando a exercises.js.

Resumen para el oral

“exercises.js es la tabla de ejercicios de nuestra base de datos simulada: un array con nombre, grupo muscular, video y descripción de cada ejercicio, más un helper getExerciseById para buscarlos por id. Las rutinas (routines.js) no copian esta información — cada item de una rutina solo guarda un exerciseId como clave foránea. Esto significa que cualquier pantalla que necesita mostrar el ejercicio completo dentro de una rutina (ver_día_x_user.js, editar_rutina_trainer.js, mi_rutina_user.js) tiene que hacer el 'join' manualmente llamando getExerciseById, evitando así duplicar datos entre rutinas distintas que comparten los mismos ejercicios.”